Artificial Intelligence

BS (CS) _SPRING_2026

Lab_10 Manual

Learning Objectives:

1. Understand and implement Adversarial Search using the Alpha-Beta Pruning technique.
2. Enhance the efficiency of game-playing agents using optimization strategies in game trees.

Adversarial Search with Alpha-Beta Pruning

Adversarial search is used when more than one agent is actively competing in the same search space, such as in two-player games. Unlike traditional single-agent search algorithms that aim to find a sequence of actions to reach a goal, adversarial searches simulate competition, where each agent considers the actions of the other and their impact.

Such environments are called multi-agent environments, and adversarial search is crucial in modeling and solving problems in these domains.

Game Playing in AI

Game playing is a well-known domain for applying adversarial search techniques.

Games have:

- Clear rules
- Defined win/loss conditions
- Legal moves that can be programmatically generated

Search techniques like Breadth-First Search (BFS) or Depth-First Search (DFS) become inefficient due to the high branching factor. Instead, Minimax Search is typically used.

However, Minimax has its own limitations due to the exhaustive nature of tree traversal. This is where Alpha-Beta Pruning enhances performance by reducing unnecessary node evaluations.

Alpha-Beta Pruning:

Alpha beta pruning in Artificial Intelligence is a way of improving the minimax algorithm. In the minimax search method, the number of game states that it must investigate grows exponentially with the depth of the tree. We cannot remove the exponent, but we can reduce it by half. As a result, a technique known as pruning allows us to compute the proper minimax choice without inspecting each node of the game tree. Alpha-beta pruning may be done at any level in a tree, and it can occasionally trim the entire sub-tree and the tree leaves. It is named alpha-beta pruning in artificial intelligence because it involves two parameters, alpha, and Beta.

Condition for Alpha Beta Pruning

The two parameters of alpha beta pruning in artificial intelligence are

Alpha (α)

- It is the **best highest value** found so far along the path for the **maximizer (MAX)**.
- It represents a **lower bound** of the possible outcome.
- The initial value of alpha is: $\alpha = -\infty$

Detailed Explanation:

- Alpha is updated only at **MAX nodes** because MAX tries to **maximize the score**.
- Whenever MAX explores a child node, it compares the returned value with the current alpha and updates it: $\alpha = \max(\alpha, \text{value})$
- Alpha keeps increasing as better options are found.

Interpretation:

- Alpha tells us: “The maximizer is guaranteed at least this value.”

Example:

- If $\alpha = 5$, it means MAX already has a strategy that ensures a score of at least 5.
- Any future branch producing a value less than 5 is not useful for MAX.

Beta (β)

- It is the **best lowest value** found so far along the path for the **minimizer (MIN)**.
- It represents an **upper bound** of the possible outcome.
- The initial value of beta is: $\beta = +\infty$

Detailed Explanation:

- Beta is updated only at **MIN nodes** because MIN tries to **minimize the score**.
- Whenever MIN explores a child node, it updates beta as: $\beta = \min(\beta, \text{value})$
- Beta keeps decreasing as better (smaller) values are found.

Interpretation:

- Beta tells us: “The minimizer can force the value to be at most this.”

Example:

- If $\beta = 3$, it means MIN can ensure the outcome will not exceed 3.
- Any branch producing a value greater than 3 is not useful for MIN.

• The condition for Alpha-Beta Pruning is $\alpha \geq \beta$

Detailed Explanation:

- This is the **cutoff condition** used to stop exploring a branch.
- When:

  $\alpha \geq \beta$

  it means:

  - The maximizer already has a better or equal option (α)
  - The minimizer already has a better or equal option (β)

Why Pruning Happens:

- The current branch **cannot produce a better result** than what has already been found.
- Therefore, continuing exploration is unnecessary.

Example:

- Suppose:
  - $\alpha = 6$ (MAX can guarantee 6)
  - $\beta = 5$ (MIN can restrict to 5)
- Since:

  $6 \geq 5$

- This branch is **pruned** because it cannot improve the final decision.

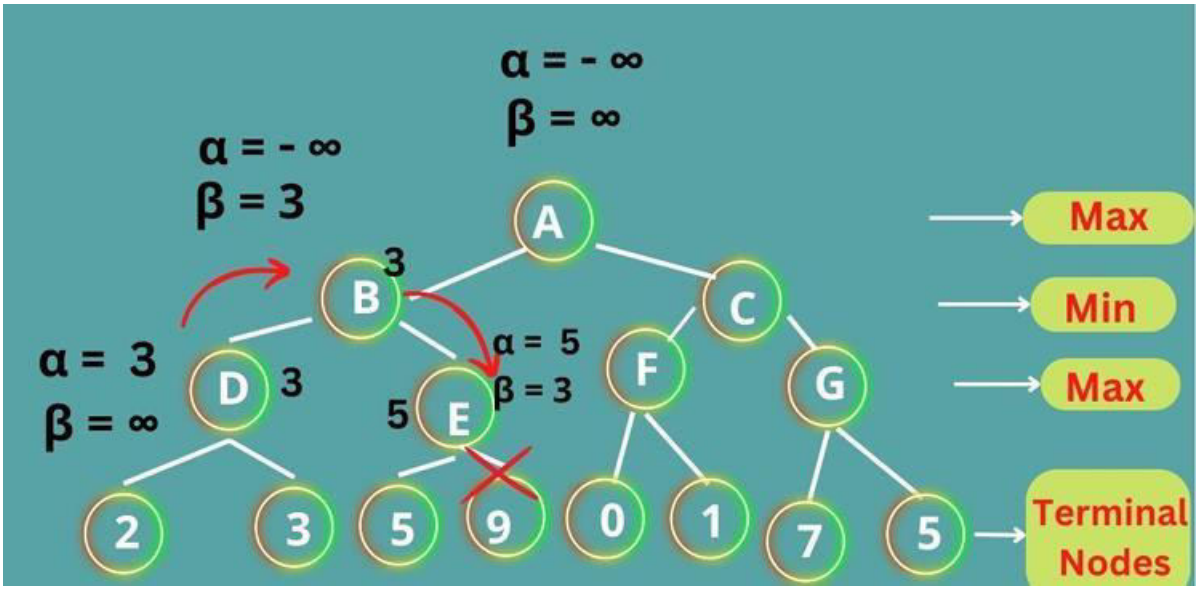
• Alpha is updated only when it's MAX's turn, and Beta is updated only when it's MIN's turn

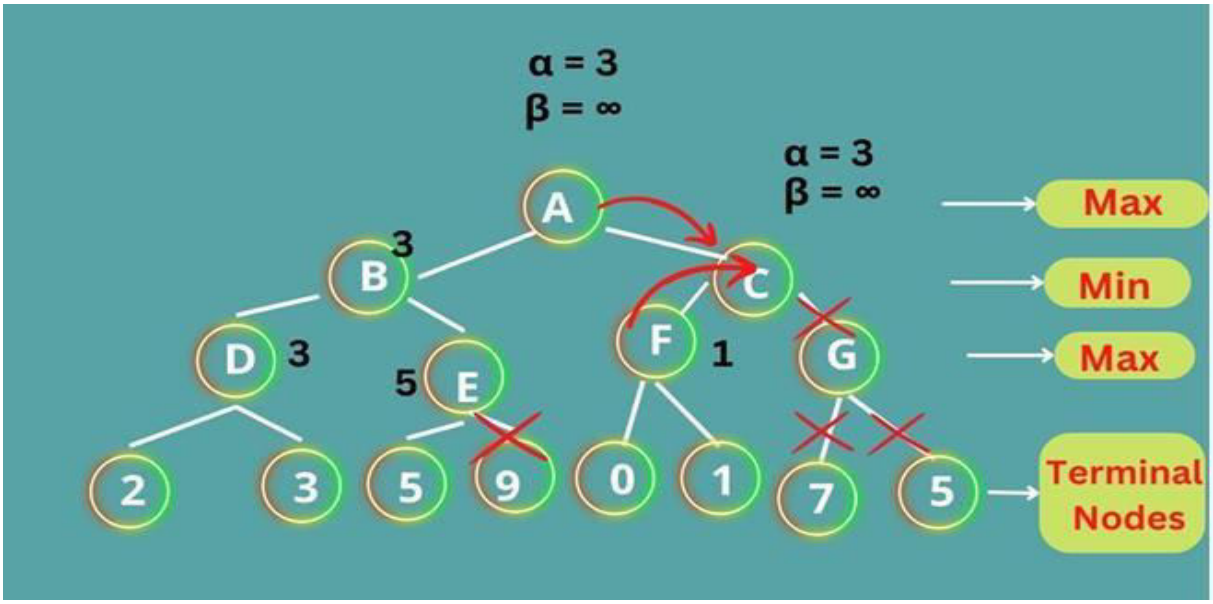
Detailed Explanation:

- At **MAX nodes**:
  - The goal is to **maximize the score**
  - So only alpha is updated
  - Beta remains unchanged
- At **MIN nodes**:
  - The goal is to **minimize the score**
  - So only beta is updated
  - Alpha remains unchanged

Reason:

- Each player only updates the bound relevant to their objective:
  - MAX improves the lower bound (alpha)
  - MIN improves the upper bound (beta)





Pseudocode for Alpha-Beta Pruning:

```
def alphabeta(node, depth, alpha, beta, maximizingPlayer):
    if depth == 0 or TERMINAL_TEST(node):
        return EVALUATION(node)

    if maximizingPlayer:
        value = -INFINITY
        for child in MOVE_GEN(node):
            value = max(value, alphabeta(child, depth-1, alpha, beta, False))
            alpha = max(alpha, value)
            if beta <= alpha:
                break  # Beta cut-off
        return value
    else:
        value = +INFINITY
        for child in MOVE_GEN(node):
            value = min(value, alphabeta(child, depth-1, alpha, beta, True))
            beta = min(beta, value)
            if beta <= alpha:
                break  # Alpha cut-off
        return value
```

Key Components

1. MOVE-GEN(state)

- Generates all possible legal moves from the current position.
- Depends on the rules of the game (e.g., chess, tic-tac-toe).
- Output: a list of successor states (children).

Example:

- Tic-Tac-Toe → all empty cells
- Chess → all legal piece moves

Important point:

- Move ordering significantly affects performance. Exploring better moves first increases pruning efficiency.

2. EVALUATION(state)

- Assigns a numerical score to a game state.
- Always evaluated from the maximizing player's perspective.

Typical scoring:

- Win → $+\infty$ (or very large value)
- Loss → $-\infty$
- Draw → 0
- Non-terminal states → heuristic values

Example (Chess):

- Material balance (queen = 9, rook = 5, etc.)
- Board control, mobility, king safety

Important point:

- Used when:
  - The depth limit is reached, or
  - The state is not terminal but further search is stopped

3. TERMINAL-TEST(state)

- Determines whether the game has ended:
  - Win
  - Loss
  - Draw

Returns:

- true → stop recursion
- false → continue searching

Important point:

- Prevents unnecessary exploration and ensures correct termination.

4. Alpha (α) and Beta (β)

These values control pruning:

Symbol	Meaning
Alpha (α)	Best (highest) value found so far for the maximizer
Beta (β)	Best (lowest) value found so far for the minimizer

Interpretation:

- Alpha is a lower bound (maximizer seeks higher values)
- Beta is an upper bound (minimizer seeks lower values)

How Alpha-Beta Works (Step-by-Step)

Step 1: Initialization

- Start at the root node with:
  - alpha = $-\infty$
  - beta = $+\infty$

This means no constraints are applied initially.

Step 2: Generate Moves

- Use MOVE-GEN(state) to produce all possible successor states.

Step 3: Recursive Exploration

The algorithm alternates between two types of nodes:

Maximizing node:

- Goal: maximize value
- Update rule:

```
alpha = max(alpha, value)
```

Minimizing node:

- Goal: minimize value
- Update rule:

```
beta = min(beta, value)
```

Step 4: Propagate Bounds

- Pass updated alpha and beta values to deeper recursive calls.
- These bounds represent constraints from previously explored branches.

Step 5: Pruning Condition

Cutoff rule:

- If alpha ≥ beta, stop exploring the current branch.

Why Pruning Works

At a maximizing node:

- If the current value becomes ≥ beta:
  - The minimizer will avoid this branch.
  - Remaining children are pruned.

At a minimizing node:

- If the current value becomes ≤ alpha:
  - The maximizer already has a better option.
  - Remaining children are pruned.

Step 6: Backtracking

- Each recursive call returns a value.
- The parent node updates its best value accordingly.

Step 7: Final Decision

- At the root, select the move that leads to the best guaranteed outcome.